# Tu trabajo dentro del proyecto

**Leo, tu responsabilidad:** establecer qué registros cumplen las reglas, explicar los rechazos y guardar resultados de forma consistente.

## ¿Qué construimos entre los tres?

Un pipeline ETL es una secuencia de pasos para extraer datos, transformarlos y cargarlos en una base. Aquí recibe registros de cajeros ATM y transferencias digitales. Un registro es una fila u objeto que describe una operación; un lote es el conjunto que procesamos en una ejecución.

El sistema mide problemas, aplica reglas, guarda aceptadas en **transacciones_limpias** y conserva inválidas en **cuarentena_errores**. Omitir un duplicado significa registrar que ya había una copia, no perderlo sin explicación.

| Rol | Qué entrega al equipo |
|---|---|
| Maki / ingeniería 1 | Diagnóstico, faltantes, duplicados e IQR. |
| Avilix / ingeniería 2 | Lectura, integración, divisas, catálogo, MinMax y conexión del flujo. |
| Leo / ingeniería 3 | Modelos Pydantic, errores detallados, persistencia y pruebas. |

## Archivos que debes reconocer

**src/validation.py:** modelos ATMTransaction y DigitalTransaction, validadores y captura de errores.
**src/persistence.py:** tablas SQLite y guardado transaccional.
**tests/test_validation_and_persistence.py:** pruebas de reglas y persistencia.

> Esta guía explica el ETL obligatorio de los puntos 4 y 6 del profesor. No necesitas explicar dashboard ni simuladores. Los módulos se integraron y adaptaron en conjunto: el rol indica responsabilidad técnica, no autoría exclusiva de cada línea.

# 1. Validar con Pydantic

**Validar** es comprobar que el dato cumple un contrato. Pydantic permite definir ese contrato con BaseModel. Cada modelo especifica campos y tipos. Un field_validator es una función que comprueba o normaliza un campo al construir el modelo.

| Elemento | Regla y propósito |
|---|---|
| ID y categorías | Se normalizan cadenas y se verifican valores admitidos. |
| Monto | Debe ser finito; una operación monetaria requiere valor positivo. Consulta de saldo permite cero. |
| Comisión ATM | No negativa. Un faltante permitido puede pasar a imputación. |
| Divisa digital | MXN o USD. |
| CLABE | Texto de exactamente 18 dígitos ASCII; conserva ceros iniciales. |
| Correo | Opcional; si está presente debe cumplir formato de correo. |
| Fecha | Formato interpretable y reglas de zona horaria por fuente. |

Un faltante permitido no es lo mismo que un formato corrupto. Un monto ausente puede estimarse después; un texto que no representa un número debe rechazarse. También se impide tratar valores no finitos como dinero válido.

> No afirmes que verificamos la existencia de una dirección de correo ni el dígito de control de la CLABE: aquí se comprueba su formato. El catálogo valida la existencia de la cuenta interna del proyecto.

Los modelos de entrada normalizan y permiten conversiones controladas; no usan strict=True global y omiten campos extra. El modelo AnaliticaFinal sí usa validación estricta para los resultados calculados. Explica esa diferencia sin decir que todo el pipeline es estricto de la misma manera.

# 2. Capturar y explicar errores

Pydantic produce ValidationError cuando falla una regla. Las funciones de validación detallada lo capturan y convierten sus detalles en un resultado de rechazo. Así una fila inválida no obliga a abandonar las demás filas del archivo.

```
try:
    modelo = Modelo.model_validate(registro)
except ValidationError as error:
    # Convertir el error en motivos y campos afectados.
    # El pipeline conserva el registro en cuarentena.
```

El fragmento muestra la idea; los nombres reales son ATMTransaction, DigitalTransaction y las funciones validate_*_record_detailed de src/validation.py.

| Campo de cuarentena | Para qué sirve |
|---|---|
| original_record | Conserva lo que llegó, para investigar. |
| codigo_error | Clasifica el problema. |
| motivo_rechazo_cuarentena | Explica el motivo en texto. |
| detalles_errores | Permite guardar varios fallos y campos. |
| fase | Indica en qué etapa fue rechazado. |
| estado_seguimiento | Permite seguir su atención posterior. |

**Ejemplo real:** ATM-TX-00009 referencia CTA-8997. La cuenta no está en el catálogo, por eso queda CUENTA_NO_REGISTRADA en INTEGRACION. En este caso campo_afectado puede ser null; el detalle del error identifica id_cuenta. No todos los rechazos provienen de Pydantic.

> Una operación con estado bancario RECHAZADA puede estar en transacciones_limpias si sus datos son válidos. “Limpia” describe calidad del registro; no significa que el banco aprobó el movimiento. SPEI-2026-00003 demuestra esa diferencia.

# 3. Guardar con SQLite

SQLite es una base de datos almacenada en un archivo. Una tabla organiza registros en filas y columnas. El archivo de entrega es entrega/pipeline_banco.sqlite; abrirlo como texto normal no permite leerlo correctamente.

| Tabla o grupo | Qué conserva |
|---|---|
| transacciones_limpias | Operaciones aceptadas y payload JSON completo. |
| cuarentena_errores | Registros rechazados y explicación. |
| ejecuciones | Identidad y estado de cada procesamiento. |
| evidencia_lotes | Perfiles y parámetros de la ejecución. |
| Catálogo y versiones | Referencia de cuentas usada en el cruce. |
| Omitidos y otros registros de control | Trazabilidad de repeticiones y reprocesos. |

## Qué significa transacción en la base

Aquí una transacción SQL es un conjunto de escrituras que se confirma junto. No es lo mismo que una transferencia bancaria. **commit** confirma las escrituras; **rollback** deshace las de la transacción fallida.

Primero se registra la ejecución EN_PROCESO. Luego las escrituras del lote se realizan de forma atómica. Si terminan bien, se confirman. Si fallan, se revierten y queda evidencia de ejecución FALLIDA. No deben quedar algunas operaciones del lote guardadas y otras perdidas por un fallo intermedio.

> payload es texto JSON con el registro completo. Los importes Decimal se serializan como cadenas decimales para conservar precisión. Las columnas externas permiten buscar ID, cuenta, fuente y ejecución sin interpretar todo el payload.

La combinación fuente + ID evita repetir una identidad. La huella distingue repetición del mismo contenido de un conflicto. La identidad de carga también utiliza hashes y versiones para reconocer una ejecución ya procesada.

# 4. Resiliencia y pruebas

**Resiliencia** significa manejar errores de forma controlada y conservar evidencia. No significa ocultar excepciones ni prometer que un fallo de disco nunca interrumpirá el programa.

| Situación | Resultado esperado |
|---|---|
| Registro válido | Se acepta y conserva sus campos normalizados. |
| CLABE o correo inválido | Cuarentena con detalle; otras filas continúan. |
| Cuenta no registrada | Cuarentena en integración. |
| Duplicado de mismo contenido | Se reconoce y se registra la omisión. |
| Misma identidad, distinto contenido | Se trata como conflicto. |
| Fallo durante escritura del lote | Rollback y ejecución FALLIDA. |

## La prueba de rollback, paso a paso

La prueba test_batch_rolls_back_and_records_failed_execution crea una base temporal y un trigger que provoca un fallo al insertar ATM-SECOND. Intenta insertar ATM-FIRST y ATM-SECOND en el mismo lote. Después comprueba que quedaron **cero filas limpias** y una ejecución FALLIDA.

La prueba espera sqlite3.IntegrityError: la excepción se propaga deliberadamente. Lo comprobado es que no queda una carga parcial y que existe evidencia del fallo. No se demuestra que todas las excepciones de infraestructura sean absorbidas.

## Ejecutar las pruebas

```
.venv\Scripts\python.exe -m pytest -q
# Solo validación y persistencia:
.venv\Scripts\python.exe -m pytest -q tests/test_validation_and_persistence.py
```

Ejecuta desde la raíz Pipeline_Banco, con el entorno del proyecto instalado. La evidencia previa entrega/pruebas.txt registra 50 pruebas aprobadas. Ese es el resultado guardado de la batería completa; una ejecución nueva muestra su propio resultado.

# 5. Revisar la base y practicar

```
.venv\Scripts\python.exe entrega/guias/consultar_ejemplos.py leo
```

El consultor abre la base oficial en modo de solo lectura. Muestra conteos, el rechazo real y controles de integridad. integrity_check debe responder ok; foreign_key_check sin filas significa que no encontró referencias incumplidas.

| Fuente | Leídas | Limpias | Cuarentena | Omitidas |
|---|---|---|---|---|
| ATM | 620 | 568 | 32 | 20 |
| DIGITAL | 600 | 504 | 96 | 0 |
| TOTAL | 1,220 | 1,072 | 128 | 20 |

Comprueba: 1,072 + 128 + 20 = 1,220. Los motivos incluyen 56 incidencias de cuenta, 46 de CLABE y 30 de correo. Suman 132 porque una fila puede tener más de un motivo; no son 132 filas rechazadas.

## Consulta para un visor SQLite

```
SELECT COUNT(*) AS limpias FROM transacciones_limpias;
SELECT COUNT(*) AS rechazadas FROM cuarentena_errores;
SELECT id_transaccion,
 json_extract(payload, '$.estado_operacion') AS estado
FROM transacciones_limpias
WHERE id_transaccion = 'SPEI-2026-00003';
```

Si un campo no aparece, inspecciona el payload y usa su nombre exacto del contrato: el SQL no inventa campos ausentes. El consultor muestra el registro completo para comprobarlo.

## Ejercicio mental y conexión con el equipo

En un lote didáctico de 10 filas, 7 válidas, 2 inválidas y 1 copia adicional deben quedar 7 aceptadas, 2 en cuarentena y 1 omitida. Maki determina calidad y tratamiento; Avilix integra y calcula; tú recibes los resultados y garantizas guardado y trazabilidad.

# Cómo explicarlo con tus palabras

## Una explicación breve

“Mi parte valida los registros con Pydantic y convierte los fallos en motivos de cuarentena. Guarda los resultados en SQLite usando commit y rollback. Las pruebas verifican reglas, duplicados y que un fallo de escritura no deje el lote parcialmente guardado.”

## Preguntas que debes poder responder

**¿Por qué no borrar inválidas?**
Cuarentena permite explicar y revisar el problema sin contaminar las aceptadas.

**¿Una fila limpia fue una operación exitosa?**
No. Calidad de datos y estado bancario son conceptos distintos.

**¿Qué evita rollback?**
Que un fallo de escritura deje persistido solo un fragmento del lote.

**¿Qué garantiza una prueba aprobada?**
El comportamiento del caso probado; no demuestra ausencia de todos los errores posibles.

## Comprueba que lo entiendes

Explica un ejemplo sin leer, señala la función que lo realiza y encuentra su resultado en la base. Distingue un dato observado, una estimación y una regla de negocio. Si puedes ejecutar el ejemplo y justificarlo, ya tienes una base para defenderlo.

## Fuentes de esta guía

Guía del profesor: GUIA_LINEAMIENTOS_PROYECTO_ALUMNOS.pdf, puntos 4 y 6, páginas 2 y 3. Código: src/validation.py, src/persistence.py, tests/test_validation_and_persistence.py. Evidencia: entrega/pipeline_banco.sqlite, entrega/resultados.json y entrega/pruebas.txt. Las cifras corresponden a la base entregable original.